My Name: Siddharth Puhan

Task: Week 4 – Technical Learning & Research

Week 4 – Technical Learning & Research Report

> *Note: code snippets, workflows and schemas in this report are illustrative examples written to explain each assigned topic. They have not been run or tested as part of this report, so no recorded outputs or results are claimed.*

This week's topics connect the parts of a full-stack AI application: a Flask backend that calls the OpenAI API, an Angular frontend that presents AI responses, user authentication with JWT, Angular route guards and interceptors, and MongoDB for data storage. Each topic has a **Learning** part (core concepts with an illustrative example) and a **Research** part (a comparison or deeper look at one question).

Learning objectives

- **Backend AI integration:** build a secure Flask gateway that calls the OpenAI API using a protected key.
- **Frontend AI interface:** build an Angular interface with async calls, input validation and loading states.
- **Backend authentication:** compare session and token authentication, implement JWT in Flask, and understand where OAuth 2.0 fits.
- **Frontend authentication:** manage login state in a single-page app with route guards and interceptors, and review token storage options.
- **NoSQL storage:** model documents in MongoDB, understand BSON and ObjectId, connect with PyMongo, and compare SQL with NoSQL.

1. Backend → OpenAI

1.1 Learning: OpenAI API Integration

Core concepts

A **large language model (LLM)** is a neural network trained on a very large amount of text. It generates a reply one token at a time, each time predicting the most likely next token from the context so far. The **OpenAI API** gives access to these models over HTTPS, so we send JSON requests and receive JSON responses without hosting any models or GPUs ourselves.

Every request is authenticated with a secret **API key** (it starts with `sk-`), sent in the header `Authorization: Bearer <KEY>`. The key must stay private, which is why the browser should never call OpenAI directly. Instead, the Flask backend acts as a gateway: it receives the user's prompt, validates it, calls OpenAI using the key stored on the server, and returns clean JSON to the frontend.

Request, response and prompt roles

A chat request is an HTTP POST to `/v1/chat/completions` with a JSON body containing the model, the messages, and settings such as `temperature` and `max_tokens`. A successful call returns HTTP 200 with the generated text and token usage details. Each message in the request has a role:

Role	Purpose
system	Sets the assistant's behaviour, tone or formatting rules.
user	The prompt entered by the end user at run time.
assistant	Earlier replies from the model, sent back to keep multi-turn context.

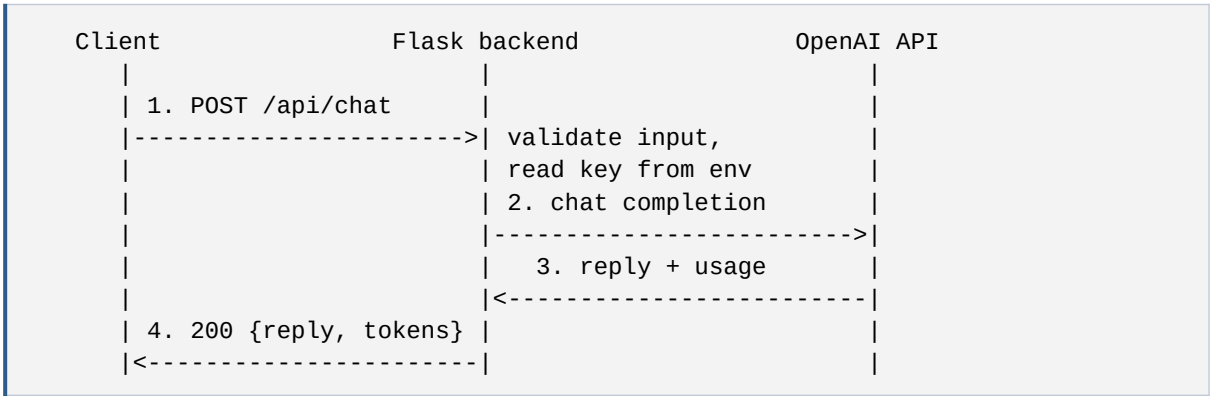
Models and parameters

- `gpt-4o` is the larger general model; `gpt-4o-mini` is cheaper and faster for simple, high-volume tasks.
- `temperature` controls randomness. Lower values give more focused, repeatable answers; higher values give more varied ones.
- `max_tokens` limits the length of the reply. Newer models use the parameter name `max_completion_tokens`, so check the model's documentation.

Keeping the key safe and handling errors

The key is stored in a `.env` file that is listed in `.gitignore`, then loaded at run time with `python-dotenv` and read through `os.environ`. Production systems normally use a proper secret manager instead. The backend should also expect failures: HTTP 401 means a bad key, 429 means a rate limit or quota was hit, and 500/503 mean a server problem. For 429 and 5xx errors the usual approach is to retry with exponential backoff and a small random delay (jitter).

Backend communication workflow



The backend works as a gateway, so the client application never holds the OpenAI key.

Illustrative Flask route

Install the packages first: `pip install flask flask-cors python-dotenv openai`.

```
# app.py (illustrative Flask route that calls OpenAI)
import os
from flask import Flask, request, jsonify
from flask_cors import CORS
from dotenv import load_dotenv
from openai import OpenAI, OpenAIError, RateLimitError

load_dotenv()  # reads OPENAI_API_KEY from the .env file

app = Flask(__name__)
CORS(app, origins=["http://localhost:4200"])
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))
```

Then the route that validates the prompt and calls the model:

```
@app.route("/api/chat", methods=["POST"])
def chat():
    data = request.get_json(silent=True) or {}
    prompt = data.get("prompt", "").strip()
    if not prompt or len(prompt) > 2000:
        return jsonify(
            {"error": "Prompt required (max 2000 characters)"}
        ), 400

    try:
        res = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role": "system",
                 "content": "You are a concise technical assistant."},
                {"role": "user", "content": prompt},
            ],
            temperature=0.3,
            max_tokens=300,
        )
        return jsonify({
            "success": True,
            "reply": res.choices[0].message.content,
            "tokens": res.usage.total_tokens,
        }), 200
    except RateLimitError:
        return jsonify({"error": "Rate limit exceeded, retry later"}), 429
    except OpenAIError:
        return jsonify({"error": "AI service error, try again"}), 502
```

How it works. The route checks the prompt length, reads the API key from the environment, and sends the request to `gpt-4o-mini` with limits on temperature and tokens. Rate-limit errors and other OpenAI errors are caught separately. A generic message is returned instead of the raw error text, so internal details are not exposed to the browser.

1.2 Research: OpenAI API Modes

OpenAI offers three main ways to interact with its models. They differ in protocol, in who keeps the conversation state, and in the kind of application they suit.

- **Chat Completions (`/v1/chat/completions`)** is the most widely used interface for text and vision. It is stateless: the client sends the whole conversation in the `messages` array on every call. It supports normal request-response calls and streaming with server-sent events (SSE).
- **Responses (`/v1/responses`)** is OpenAI's newer unified interface for agent-style applications. It has built-in tools such as web search, file search and code interpreter that run on OpenAI's side, and it can keep conversation state on the server.
- **Realtime (`/v1/realtime`)** keeps a live, two-way session open over WebSocket or WebRTC. It is meant for low-latency voice and live multimodal use, instead of repeated HTTP requests.

Aspect	Chat Completions	Responses	Realtime
Connection	HTTP POST, optional SSE streaming	HTTP POST, optional streaming	WebSocket or WebRTC session
Conversation state	Client sends full history each call	Can be managed on the server	Held inside the live session
Tools	Function calling; client runs the tool	Built-in tools run by OpenAI	Tool calls streamed in session
Best suited for	Simple text chat backends	New projects, agent workflows	Voice and live multimodal apps

Which one suits this project?

For this week's basic text application, Chat Completions is a sensible choice for learning. It matches Flask's simple request-response model, needs no socket handling, and has the most examples available. One point to note: OpenAI states that Chat Completions remains supported, but it recommends the Responses API for new projects. A new production application should therefore consider Responses. For simple text, the change is small: a different endpoint, with `input` used in place of `messages`.

2. Frontend AI UI

2.1 Learning: AI-Powered Frontend Fundamentals

A generative AI feature has a different feel from a normal form: replies take several seconds, so the interface must stay responsive and tell the user what is happening.

Frontend and backend contract

The Angular frontend calls our own Flask backend and never OpenAI directly, so the private API key cannot be seen in the browser. The prompt is sent with an HTTP **POST**, because prompts can be long and a POST body has no practical length limit like a URL does. The request and response use a simple JSON contract:

```
Request:   { "prompt": "<text>" }
Response:  { "success": true, "reply": "<text>", "tokens": 123 }
```

What the interface needs to handle

Concern	What the frontend does
Prompt submission	Reads the textarea, trims extra whitespace and sends JSON with an async POST request.
Input validation	Checks the prompt is not empty and stays within limits (for example 3 to 2000 characters) before enabling the button.
Async calls	Uses Promises, async/await or RxJS Observables so the page stays responsive during a multi-second reply.
Loading state	Disables the submit button to prevent duplicate requests and shows a loading message or spinner.
Showing replies	Displays the text with `white-space: pre-wrap` so paragraphs, lists and code blocks keep their layout.
Error handling	Catches 4xx/5xx errors, shows a clear message and keeps the user's prompt so it can be sent again.
Usability extras	Chat-style message bubbles, a copy button, and Enter to submit with Shift+Enter for a new line.

Frontend communication flow

```
User types prompt
      |
Trim and validate (for example 3 to 2000 characters)
      |
loading = true, submit button disabled
      |
HttpClient.post('/api/chat', { prompt })
      |
  success --> show reply, loading = false
  error   --> show message, keep prompt, loading = false
```

Illustrative Angular component

The component below needs `provideHttpClient()` in `app.config.ts`. First the component and its template:

```
// prompt-box.component.ts (illustrative standalone component)
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-prompt-box',
  standalone: true,
  imports: [CommonModule, FormsModule],
  template: `
    <div class="ai-box">
      <div *ngIf="error" class="alert-err">{{ error }}</div>
      <div *ngIf="reply" class="reply-panel">
        <strong>AI:</strong>
        <p>{{ reply }}</p>
      </div>
      <form (ngSubmit)="send()">
        <textarea [(ngModel)]="prompt" name="prompt"
          [disabled]="loading" placeholder="Ask a question...">
        </textarea>
        <button type="submit" [disabled]="loading || !prompt.trim()">
          {{ loading ? 'Thinking...' : 'Submit' }}
        </button>
      </form>
    </div>`,
})
```

Then the class that sends the request:

```
export class PromptBoxComponent {
  private http = inject(HttpClient);
  prompt = '';
  reply = '';
  loading = false;
  error = '';

  send() {
    if (!this.prompt.trim() || this.loading) return;
    this.loading = true;
    this.error = '';

    this.http
      .post<{ reply: string }>('http://localhost:5000/api/chat', {
        prompt: this.prompt.trim(),
      })
      .subscribe({
        next: (res) => {
          this.reply = res.reply;
          this.prompt = '';
          this.loading = false;
        },
        error: (err) => {
          this.error = err.error?.error || 'Server error';
          this.loading = false;
        },
      });
  }
}
```

How it works. The component binds the textarea with `FormsModule`, disables the controls while a request is running, handles the asynchronous response with RxJS, and turns backend failures into a readable error message on the page.

2.2 Research: Tokens, Costs and Limits

Understanding tokens is important for building AI features that stay affordable and reliable.

How tokens and limits work

- **Tokens** are sub-word pieces of text. As a rough guide, 1 token is about 4 characters or 0.75 of an English word, so 1,000 tokens is about 750 words.
- **Input tokens** are the prompt, instructions and conversation history. **Output tokens** are the generated reply. Output tokens are priced higher (about 4 times the input price for the GPT-4o family).
- In multi-turn chat, the full history is sent every time, so input tokens grow with each turn.
- The **context window** is the most text a model can handle in one call. The **maximum output** is the most it can generate in one reply.
- **Rate limits** (requests per minute and tokens per minute, depending on the account tier) return HTTP 429 when exceeded. Budget and usage limits can also be set in the OpenAI dashboard to avoid surprise bills from a bug such as a runaway loop.

Model comparison

Model	Context window	Max output	Input / 1M	Output / 1M	Typical use
gpt-4o-mini	128,000	16,384	$0.15	$0.60	Q&A, classification, formatting, high-volume tasks
gpt-4o	128,000	16,384	$2.50	$10.00	Complex reasoning, code design, image analysis
o4-mini	200,000	100,000	$1.10	$4.40	Reasoning tasks such as maths and algorithms

Prices are in US dollars per 1 million tokens, as listed in OpenAI's documentation at the time of writing. OpenAI also offers newer model families, and prices and limits change, so the official pricing page should be checked before estimating real costs. Reasoning models can also use extra hidden reasoning tokens, which are billed as output.

Ways to reduce token usage

- **Short system prompts:** direct instructions without unnecessary wording save tokens on every call.
- **Trim the history:** keep a sliding window of recent turns (for example the last 4 to 6) instead of the whole conversation.
- **Limit the reply:** set a sensible `max_tokens` value.
- **Use prompt caching:** OpenAI applies caching automatically to prompts of 1,024 tokens or more, and cached input is billed at a lower price.
- **Route by task:** send routine questions to `gpt-4o-mini` (its input price is about 94% lower than `gpt-4o`) and use a larger model only when needed.

3. Flask JWT/Session Authentication

3.1 Learning: Authentication Concepts and JWT

Authentication basics

Authentication checks who a user is, for example with a password, a token or a digital signature. **Authorization** decides what that user is allowed to do. A failed authentication returns HTTP **401 Unauthorized**; an authenticated user without permission gets HTTP **403 Forbidden**. Because HTTP is stateless and keeps no memory between requests, every request must prove who the user is again.

Session-based and token-based authentication

Point	Session-based	Token-based (JWT)
Where state lives	On the server (memory, database or cache)	Inside a signed token held by the client
What the client sends	A session ID in a cookie	`Authorization: Bearer <token>` header
Scaling	Server must store and share sessions	No lookup needed, so it scales easily
Logging out	Delete the session on the server	Hard to revoke before the token expires

What is a JWT?

A **JSON Web Token (JWT)** is an open standard (RFC 7519) for passing claims between systems in a compact, URL-safe form. A token has three Base64URL-encoded parts joined by dots: `Header.Payload.Signature`.

Part	Contents
Header	The token type (`typ: JWT`) and signing algorithm (`alg: HS256`).
Payload	The claims, such as `sub` (user ID), `iat` (issued at), `exp` (expiry) and `role`. It is encoded, not encrypted, so secrets must never be placed in it.
Signature	A hash of the header and payload made with the secret key (HMAC-SHA256). It lets the server detect any change to the token.

Access tokens, refresh tokens and passwords

- **Access token:** short-lived (for example 15 minutes), sent in the `Authorization: Bearer` header to reach protected APIs.
- **Refresh token:** longer-lived (for example 7 days), stored securely and used at a refresh endpoint to get a new access token without logging in again.
- **Passwords:** never store them in plain text. Use a salted one-way hash. Werkzeug's `generate_password_hash` and `check_password_hash` do this (the default method depends on the Werkzeug version, for example scrypt or PBKDF2).
- **Protected endpoints:** a decorator reads the Bearer token, verifies the signature, checks that `exp` has not passed, and only then runs the route.

JWT authentication lifecycle

```
Client                          Flask backend
  |  1. POST /api/login           |
  |  {username, password}         |
  |------------------------------>| check password hash
  |                               | 2. create and sign JWT
  |  3. 200 {access_token}        |
  |<------------------------------|
  |                               |
  |  4. GET /api/protected        |
  |  Authorization: Bearer <JWT>  |
  |------------------------------>| verify signature and exp
  |  5. 200 {protected data}      |
  |<------------------------------|
```

Illustrative Flask JWT implementation

This example uses the PyJWT package (`pip install PyJWT`). First the setup, with the secret and a mock user:

```
# auth_app.py (illustrative Flask JWT example)
import os
import datetime
import jwt
from functools import wraps
from flask import Flask, request, jsonify
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
JWT_SECRET = os.environ["JWT_SECRET_KEY"]  # never hard-code the secret

# Mock user store for the example (a real app would use a database)
MOCK_DB = {
    "demo_user": {
        "id": "u101",
        "hash": generate_password_hash("demo-password"),
        "role": "engineer",
    }
}
```

Next, the decorator that protects routes:

```
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        header = request.headers.get("Authorization", "")
        if not header.startswith("Bearer "):
            return jsonify({"error": "Missing or malformed header"}), 401
        token = header.split(" ")[1]
        try:
            payload = jwt.decode(token, JWT_SECRET, algorithms=["HS256"])
        except jwt.ExpiredSignatureError:
            return jsonify({"error": "Token expired"}), 401
        except jwt.InvalidTokenError:
            return jsonify({"error": "Invalid token"}), 401
        return f(payload["sub"], *args, **kwargs)
    return decorated
```

How it works. The secret is read from an environment variable and the mock user stores only a password hash. The `token_required` decorator rejects requests with a missing, expired or invalid token, and passes the user ID from the `sub` claim to the route.

Then the login route and one protected route:

```
@app.route("/api/login", methods=["POST"])
def login():
    data = request.get_json(silent=True) or {}
    user = MOCK_DB.get(data.get("username"))
    password = data.get("password", "")
    if not user or not check_password_hash(user["hash"], password):
        return jsonify({"error": "Invalid credentials"}), 401

    now = datetime.datetime.now(datetime.timezone.utc)
    token = jwt.encode(
        {
            "sub": user["id"],
            "role": user["role"],
            "iat": now,
            "exp": now + datetime.timedelta(minutes=15),
        },
        JWT_SECRET,
        algorithm="HS256",
    )
    return jsonify({"success": True, "access_token": token}), 200


@app.route("/api/protected")
@token_required
def protected(current_user):
    return jsonify({"message": "Hello, user " + current_user})
```

How it works. The login route checks the password hash and, if it matches, signs a token that expires after 15 minutes. The protected route runs only when the decorator accepts the token.

3.2 Research: JWT vs OAuth 2.0

> **Key distinction:** JWT and OAuth 2.0 are not direct alternatives. JWT is a **token format** (RFC 7519). OAuth 2.0 is an **authorization framework** (RFC 6749). They work at different layers and are often used together.

- **JWT** defines how claims are written as JSON, encoded and signed. It describes the data structure, not how tokens are obtained.
- **OAuth 2.0** defines how an application gets permission to access resources on a user's behalf (for example with the Authorization Code flow), without the user sharing their password. It does not require a particular token format: tokens can be random opaque strings or JWTs.
- **OpenID Connect (OIDC)** is built on top of OAuth 2.0 and adds user authentication by issuing signed ID tokens in JWT format.

Aspect	JSON Web Token (JWT)	OAuth 2.0
Category	Token format	Authorization framework
Defined in	RFC 7519	RFC 6749 (bearer tokens in RFC 6750)
Purpose	Carry verifiable claims in a compact token	Let apps access resources on a user's behalf
Defines	Header, payload and signature	Roles, endpoints, grant types and token lifecycle
Validation	Signature check on the server, no lookup needed	Resource server checks the token, or asks the authorization server (introspection)
Revocation	Hard before expiry without a denylist	Revocation endpoint standardised in RFC 7009
Typical use	Session tokens between a single-page app and its API	"Sign in with Google", third-party API access, enterprise SSO

In this week's Flask example, the server issues its own JWT access tokens after a username and password login. That is simple token-based authentication, not a full OAuth 2.0 setup.

4. Angular Authentication

4.1 Learning: Angular Authentication Architecture

In a single-page app, the frontend has to remember who is logged in, control which pages can be opened, and attach the token to API calls. Angular splits these jobs across a few small pieces.

Piece	Role
`AuthService`	A singleton service (`providedIn: 'root'`) that holds the token, login and logout methods, and the login state.
Route guard	A `CanActivateFn` that runs before a route opens and redirects unauthenticated users to `/login`.
HTTP interceptor	An `HttpInterceptorFn` that clones each outgoing request to add `Authorization: Bearer <token>`.
Reactive state	Angular Signals or an RxJS `BehaviorSubject` that tell components when the login state changes.
Routing	Maps URLs to components in `app.routes.ts` without reloading the page.

Login and request flow

1. The user logs in and the backend returns an access token.
2. `AuthService` stores the token and updates the login state.
3. The guard allows protected routes because the user is authenticated.
4. The interceptor adds the Bearer token to API requests.
5. If the backend returns 401, the interceptor logs the user out and redirects to `/login` (optionally with a `returnUrl`).

> **Important:** Angular route guards only control navigation and user experience in the browser. They do **not** replace backend security. Authorization must always be enforced on the backend API for every request.

Illustrative AuthService, guard and interceptor

This example uses the functional style of modern Angular. First the service, which keeps the access token in memory:

```
// auth.service.ts
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class AuthService {
  // Access token kept in memory only (see Section 4.2)
  private token: string | null = null;

  setToken(token: string): void { this.token = token; }
  getToken(): string | null { return this.token; }
  isAuthenticated(): boolean { return this.token !== null; }
  logout(): void { this.token = null; }
}
```

The route guard:

```
// auth.guard.ts
import { inject } from '@angular/core';
import { Router, CanActivateFn } from '@angular/router';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isAuthenticated()) return true;
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url },
  });
};
```

The interceptor:

```
// auth.interceptor.ts
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { catchError, throwError } from 'rxjs';
import { AuthService } from './auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const token = auth.getToken();

  const authReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) {
        auth.logout();
        router.navigate(['/login']);
      }
      return throwError(() => err);
    })
  );
};
```

Both are then registered in the app configuration and routes:

```
// app.config.ts (excerpt)
providers: [provideRouter(routes),
            provideHttpClient(withInterceptors([authInterceptor]))]

// app.routes.ts (excerpt)
{ path: 'dashboard', component: DashboardComponent,
  canActivate: [authGuard] }
```

How it works. The guard checks the login state before a route opens and redirects to `/login` if needed. The interceptor adds the Bearer token to outgoing requests and handles 401 responses in one place. In a real app the interceptor should only add the token to requests that go to the app's own API. Because the token is kept in memory, a page refresh clears it; the refresh-token approach in Section 4.2 is one way to restore it.

4.2 Research: Token Storage and Common Threats

Where the frontend keeps a token affects how exposed it is to common web attacks. Two threats matter most:

- **Cross-Site Scripting (XSS):** malicious script runs inside the page. It can read anything JavaScript can read, such as `localStorage`.
- **Cross-Site Request Forgery (CSRF):** another site tricks the browser into sending a request that automatically includes cookies. It is reduced by `SameSite` cookies and anti-CSRF tokens.

Storage options compared

Storage	Readable by JS	XSS risk	CSRF risk	Verdict
localStorage	Yes	High: any injected script can read the token	None (header is set manually)	Avoid for sensitive tokens
sessionStorage	Yes	High: same as localStorage, cleared when the tab closes	None (header is set manually)	Avoid for sensitive tokens
In-memory (service)	Only app code	Lower: not persisted, but running script can still use it	None	Good for short-lived access tokens
HttpOnly cookie	No	Token cannot be read by scripts	Yes, needs SameSite or CSRF protection	Good for refresh tokens

Cookie attributes and recommended approach

- **HttpOnly** blocks JavaScript from reading the cookie. **Secure** sends it only over HTTPS. **SameSite=Strict** or **Lax** limits cross-site sending.
- A widely recommended pattern is to keep the short-lived access token in memory and store the long-lived refresh token in a `Secure`, `HttpOnly`, `SameSite=Strict` cookie limited to the refresh endpoint. This follows OWASP guidance to avoid storing sensitive tokens in Web Storage.
- For larger enterprise apps, the Backend-for-Frontend (BFF) pattern keeps tokens on the server side entirely.

No storage choice removes XSS itself. HttpOnly cookies stop a script from reading the token, but a script running in the page can still send requests as the user. XSS must also be prevented directly, for example with Angular's built-in output sanitisation and a Content Security Policy.

5. MongoDB Introduction

5.1 Learning: MongoDB Fundamentals and PyMongo

What MongoDB is

A database stores data so that it lasts beyond a running program and can be searched quickly. Relational (SQL) databases keep data in tables with fixed columns and strong transaction guarantees (ACID). **MongoDB** is a NoSQL **document database**: it stores flexible, JSON-like documents, which makes it easy to change the data shape as an application grows. Its server process is `mongod`, and its default storage engine is WiredTiger.

SQL term	MongoDB term
Database	Database (a container for collections)
Table	Collection (a group of documents)
Row	Document (a BSON record)
Column	Field
Primary key	`_id` field
JOIN	Embedded documents, or `$lookup`

BSON and ObjectId

MongoDB stores documents in **BSON** (Binary JSON). It supports everything JSON does, plus types such as Date, ObjectId, int32, int64, double and Binary. Each document has a unique `_id`. By default this is an **ObjectId**, a 12-byte value made of a 4-byte timestamp, a 5-byte random value and a 3-byte incrementing counter.

Basic CRUD operations

```
// mongosh (illustrative)
db.users.insertOne({ username: "demo", role: "intern", logins: 0 })
db.users.find({ role: "intern" })
db.users.updateOne({ username: "demo" },
                   { $set: { role: "engineer" }, $inc: { logins: 1 } })
db.users.deleteOne({ username: "demo" })
```

`insertOne` adds a document, `find` reads documents, `updateOne` changes fields (for example with `$set` and `$inc`), and `deleteOne` removes a document.

Schema design and indexing

- **Embedding** puts related data inside one document. It suits one-to-one and small, bounded one-to-many data, and needs only one read.
- **Referencing** stores an ObjectId that points to another document. It suits large, unbounded one-to-many and many-to-many relationships.
- **Indexes** (B-tree) make lookups fast. `_id` is indexed automatically. A query with no useful index scans the whole collection (a `COLLSCAN`), which becomes slow on large data.

Illustrative user document

```
// users collection (illustrative document)
{
  _id: ObjectId("66f4a8b2c1e8473a90f12345"),
  username: "siddharth_puhan",
  email: "siddharth@example.com",
  passwordHash: "scrypt:32768:8:1$<salt>$<hash>",  // placeholder value
  role: "intern",
  isActive: true,
  createdAt: ISODate("2026-11-09T18:00:00Z")
}
```

User documents must never store plain-text passwords. Only a salted hash is stored, as in the `passwordHash` field above.

Illustrative Flask and PyMongo integration

Flask connects to MongoDB with the `pymongo` driver, using a connection URI such as `mongodb://localhost:27017/`.

```
# mongo_demo.py (illustrative Flask + PyMongo example)
from flask import Flask, request, jsonify
from pymongo import MongoClient, ASCENDING
from werkzeug.security import generate_password_hash

app = Flask(__name__)
client = MongoClient("mongodb://localhost:27017/")
db = client["internship_week4_db"]
db.users.create_index([("username", ASCENDING)], unique=True)


@app.route("/api/users/register", methods=["POST"])
def register():
    data = request.get_json(silent=True) or {}
    username = data.get("username", "").strip()
    password = data.get("password", "")

    if not username or len(password) < 8:
        return jsonify(
            {"error": "Username and 8+ character password required"}
        ), 400
    if db.users.find_one({"username": username}):
        return jsonify({"error": "Username already exists"}), 409

    res = db.users.insert_one({
        "username": username,
        "passwordHash": generate_password_hash(password),
        "role": "user",
    })
    return jsonify(
        {"success": True, "insertedId": str(res.inserted_id)}
    ), 201
```

How it works. The unique index on `username` stops duplicate accounts, the password is hashed with a salt before saving, and the `ObjectId` is converted to a string so it can be returned as JSON.

5.2 Research: NoSQL vs SQL

SQL databases and document databases solve similar problems in different ways. The table summarises the main differences.

Aspect	Relational SQL (PostgreSQL, MySQL)	Document NoSQL (MongoDB)
Data model	Tables with rows and columns	Hierarchical BSON/JSON documents
Schema	Fixed schema; changes need migrations	Flexible; documents can differ in shape
Relationships	Foreign keys and JOINs across tables	Embedded documents, or `$lookup` between collections
Query language	Declarative SQL	JSON-based Query API and aggregation pipeline (`$match`, `$group`, `$project`)
Transactions	Full ACID by default	Multi-document ACID transactions supported since version 4.0
Scaling	Usually vertical scaling plus read replicas	Built-in horizontal scaling with sharding
Typical use	Financial ledgers, ERP, data with many joins	Content management, IoT, mobile apps, AI chat logs

Neither is better in every case. Relational databases are a strong fit when data has many relationships and strict consistency is the priority, such as accounting records. Document databases are useful for fast prototyping, nested content and logging, such as storing AI conversation history. Many real systems use both, which is called polyglot persistence.

6. Overall Learning Summary

The five topics fit together as one application. The Angular app talks only to the Flask backend. The backend checks the user's token, validates input, and then uses MongoDB for data and OpenAI for AI replies.

```
Angular SPA (route guard + HTTP interceptor)
        |   HTTPS + Bearer JWT
        v
Flask REST backend (JWT check, input validation)
    |                           |
    v                           v
MongoDB                    OpenAI API
(users, chat logs)         (Chat Completions)
```

Key points to remember

- Passwords are stored only as salted hashes.
- API keys and JWT secrets stay on the server in environment variables.
- Route guards improve the user experience, but the backend must enforce authorization on every request.
- Managing tokens (short prompts, trimmed history, the right model) keeps AI costs predictable.

7. References

1. **OpenAI:** API documentation; Pricing; Responses vs Chat Completions
2. **Flask:** Flask documentation; Flask-CORS documentation
3. **Angular:** Routing; HTTP interceptors
4. **OWASP:** Cheat Sheet Series (Session Management, HTML5 Security and JSON Web Token cheat sheets)
5. **MongoDB:** MongoDB Manual; PyMongo documentation
6. **IETF:** RFC 7519 (JSON Web Token); RFC 6749 (OAuth 2.0); RFC 6750 (Bearer Token Usage); RFC 7009 (Token Revocation)